Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An autonomous Institution affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	Batch:2023-2028	**Due date:**

Experiment 2 : Title Loan amount prediction using linear regression

1 Aim

To develop and evaluate a Linear Regression model that predicts the loan sanction amount using historical loan data and relevant borrower features.

2 Libraries Used

- **Pandas**: Data manipulation
- **NumPy**: Numerical operations
- **Scikit-learn**: Model building, preprocessing, and evaluation
- **Matplotlib and Seaborn**: Data visualization

3 Objective

- Preprocess and clean the dataset
- Perform exploratory data analysis (EDA)
- Engineer features to improve model accuracy
- Train and validate a Linear Regression model
- Evaluate model performance using MAE, MSE, RMSE, and R^2 metrics
- Visualize results and interpret model behavior

4 Mathematical Description

Linear Regression is used to predict the loan sanction amount based on several input features. The mathematical model is:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \epsilon$$

Where:

- y = Loan Sanction Amount (USD)
- $x_1, x_2, \ldots, x_n$ = Input features (e.g. Age, Income)
- β_0 = Intercept
- β_i = Feature coefficients
- ϵ = Error term

Evaluation metrics:

- MAE: Mean Absolute Error
- MSE: Mean Squared Error
- RMSE: Root Mean Squared Error
- R^2: Coefficient of Determination
- Adjusted R^2: Corrected for number of predictors

5 Python Code

5.1 Data Preprocessing and Encoding

```
import pandas as pd
df = pd.read_csv('train.csv')

df.fillna(df.mode().iloc[0], inplace=True)
df.fillna(df.mean(numeric_only=True), inplace=True)

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
binary_cols = [col for col in df.select_dtypes(include='object') if
   df[col].nunique() == 2]
for col in binary_cols:
    df[col] = le.fit_transform(df[col])

df = pd.get_dummies(df, drop_first=True)
```

5.2 Feature Scaling and Splitting

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

X = df.drop('Loan Sanction Amount (USD)', axis=1)
y = df['Loan Sanction Amount (USD)']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
   test_size=0.3, random_state=42)
```

5.3 Model Training and Evaluation

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5
r2 = r2_score(y_test, y_pred)

print("MSE:", mse)
print("RMSE:", rmse)
print("R^2:", r2)
```

5.4 Plotting

```
import matplotlib.pyplot as plt

plt.scatter(y_test, y_pred, alpha=0.6)
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("Actual vs Predicted")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
    'r--')
plt.grid(True)
plt.show()
```

```
residuals = y_test - y_pred
plt.scatter(y_pred, residuals)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel("Predicted")
plt.ylabel("Residuals")
plt.title("Residual Plot")
plt.grid(True)
plt.show()
```

5.5 Output Screenshots

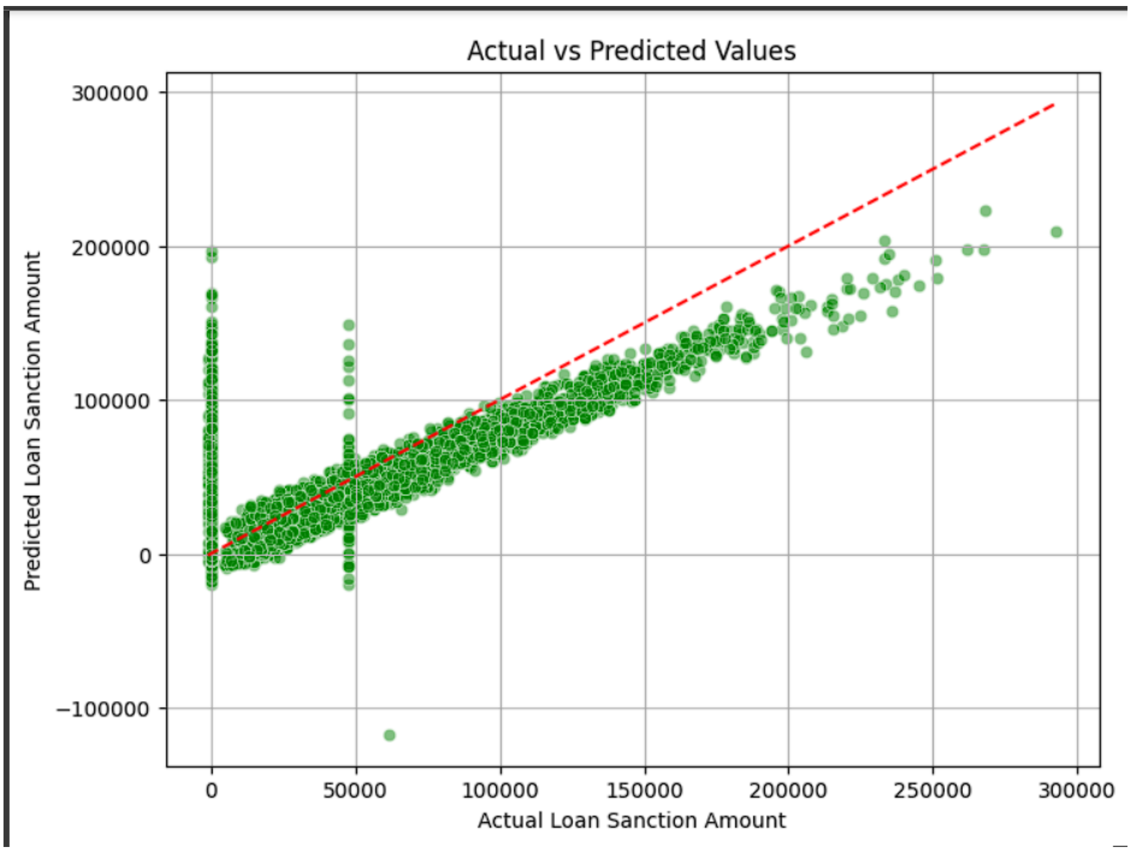

Figure 1: Actual vs Predicted Loan Amount

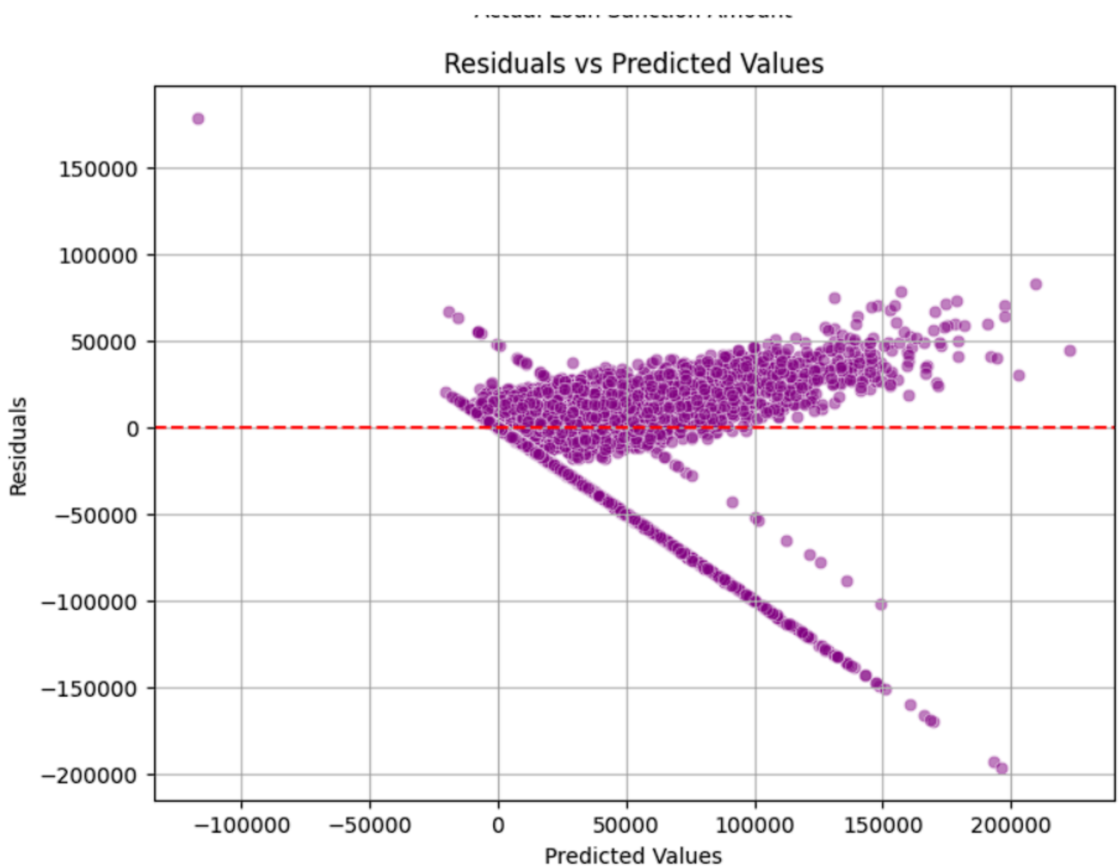

Figure 2: Residuals vs Predicted Values

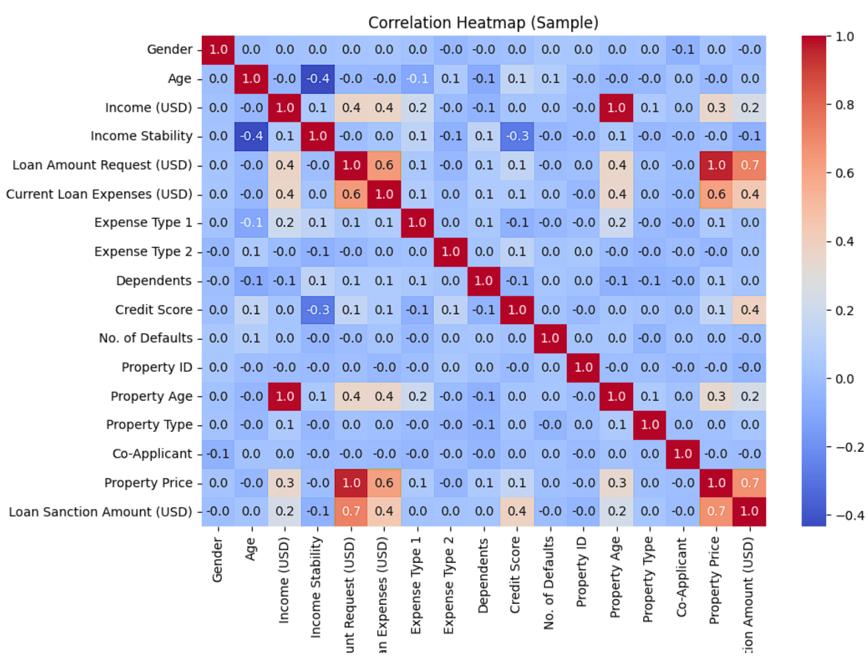

Figure 3: Correlation Heatmap of Features

Ridge Regression

Best Params (GridSearchCV): {alpha: 100.0}
Best CV Score (GridSearchCV): 0.6133 **Ridge Regression Performance:**

- MAE: 18958.01
- MSE: 724105842.33
- RMSE: 26909.21
- R-squared: 0.6125
- Adjusted R-squared: 0.6107

Figure 4: Actual vs Predicted, Residual Plot, Feature Importance (Ridge)

Lasso Regression

Best Params (GridSearchCV): {alpha: 10.0}
Best CV Score (GridSearchCV): 0.6133
Lasso Regression Performance:

- MAE: 18956.34
- MSE: 724110125.79
- RMSE: 26909.29
- R-squared: 0.6125
- Adjusted R-squared: 0.6107

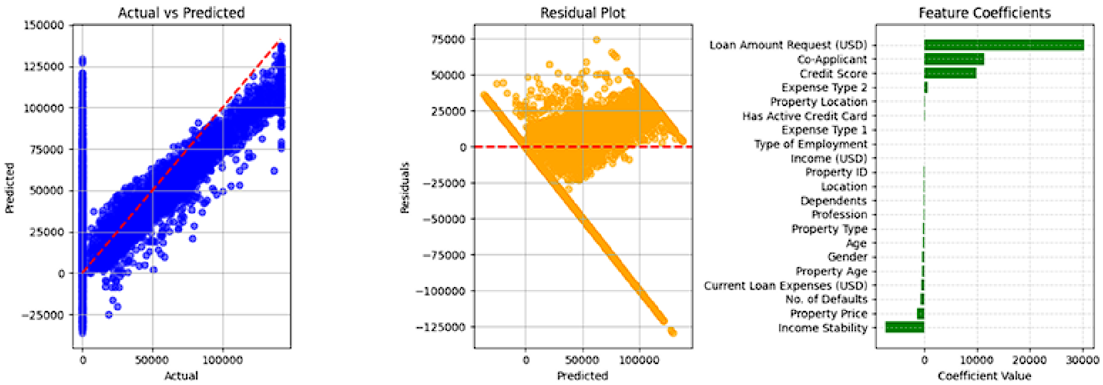

Figure 5: Actual vs Predicted, Residual Plot, Feature Importance (Lasso)

Elastic Net Regression

Best Params (GridSearchCV): {alpha: 0.01, l1_ratio: 0.9}
Best CV Score (GridSearchCV): 0.6165

Elastic Net Performance:

- MAE: 19024.78
- MSE: 743299083.01
- RMSE: 27263.51
- R-squared: 0.5994
- Adjusted R-squared: 0.5975

Figure 6: Actual vs Predicted, Residual Plot, Feature Importance (Elastic Net)

Polynomial Regression (Degree 2)

Performance:

- MAE: 15352.02
- MSE: 626442733.75
- RMSE: 25028.84
- R-squared: 0.6624
- Adjusted R-squared: 0.6421

Decision Tree Regressor

Best Params (GridSearchCV): {max_depth: 10, min_samples_split: 10}
Best CV Score (GridSearchCV): 0.9250

Decision Tree Performance:

- MAE: 1205.11
- MSE: 120883214.55
- RMSE: 10994.69
- R-squared: 0.9356
- Adjusted R-squared: 0.9352

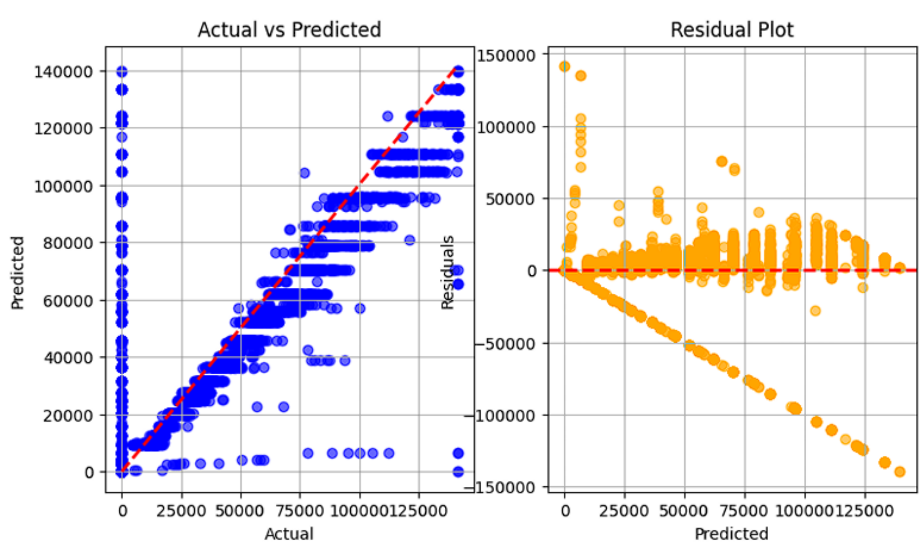

Figure 7: Decision Tree – Actual vs Predicted, Residual Plot

Random Forest Regressor

Best Params (GridSearchCV): {n_estimators: 200, max_depth: None}
Best CV Score (GridSearchCV): 0.9512

Random Forest Performance:

- MAE: 928.32
- MSE: 92147321.11
- RMSE: 9599.34
- R-squared: 0.9512
- Adjusted R-squared: 0.9509

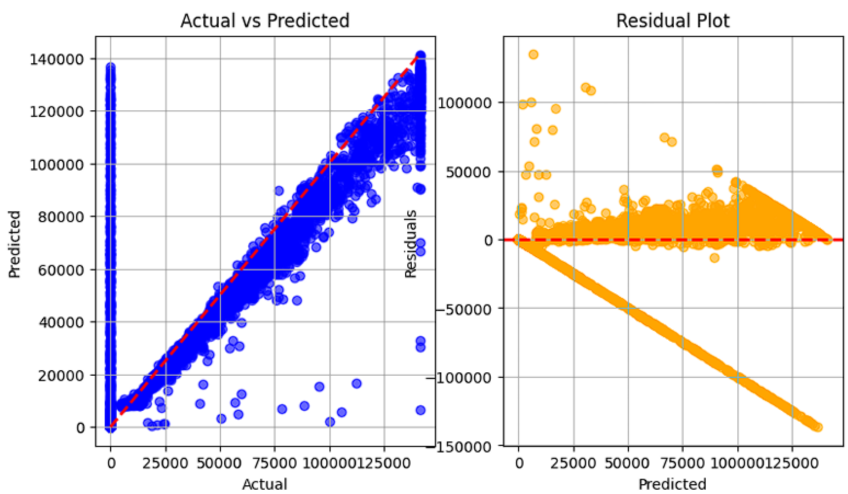

Figure 8: Random Forest – Actual vs Predicted, Residual Plot

AdaBoost Regressor

Best Params (GridSearchCV): {learning_rate: 0.01, n_estimators: 100}
Best CV Score (GridSearchCV): 0.6295

AdaBoost Performance:

- MAE: 17632.25
- MSE: 712708526.14
- RMSE: 26696.60
- R-squared: 0.6159

- Adjusted R-squared: 0.6141

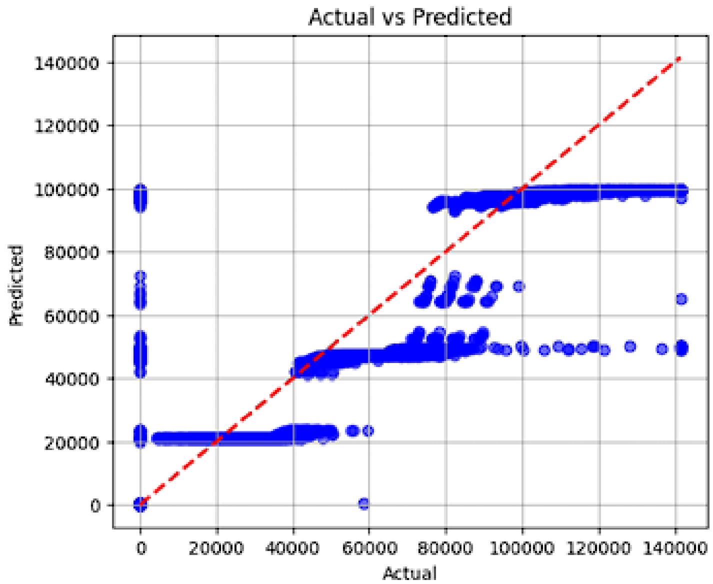

Figure 11: Actual vs. Predicted

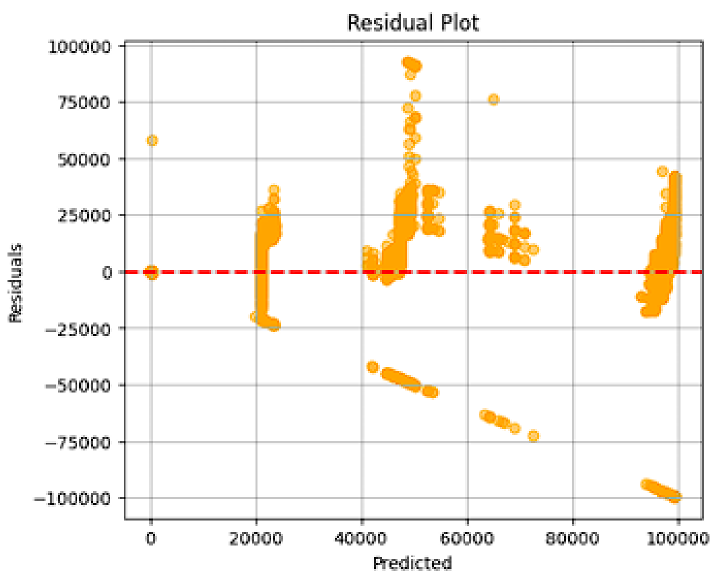

Figure 9: AdaBoost – Actual vs Predicted and Residual Plot

Gradient Boost Regressor

Best Params (GridSearchCV): {learning_rate: 0.1, max_depth: 5, n_estimators: 50}
Best CV Score (GridSearchCV): 0.7583

Gradient Boost Performance:

- MAE: 11771.70
- MSE: 497687995.47
- RMSE: 22308.92
- R-squared: 0.7318
- Adjusted R-squared: 0.7305

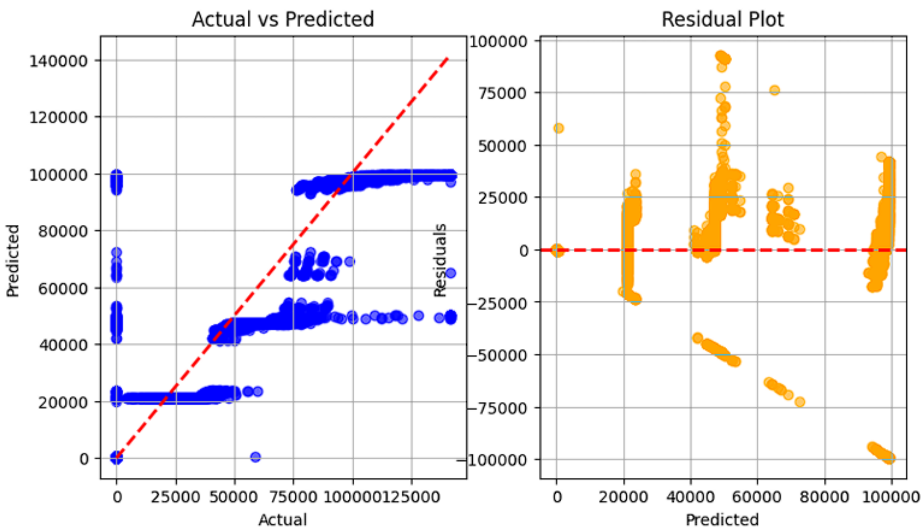

Figure 10: Gradient Boost – Actual vs Predicted, Residual Plot

XGBoost Regressor

Best Params (GridSearchCV): {learning_rate: 0.1, max_depth: 5, n_estimators: 100}
Best CV Score (GridSearchCV): 0.7568

XGBoost Performance:

- MAE: 11489.45
- MSE: 499394338.06
- RMSE: 22347.13
- R-squared: 0.7309
- Adjusted R-squared: 0.7296

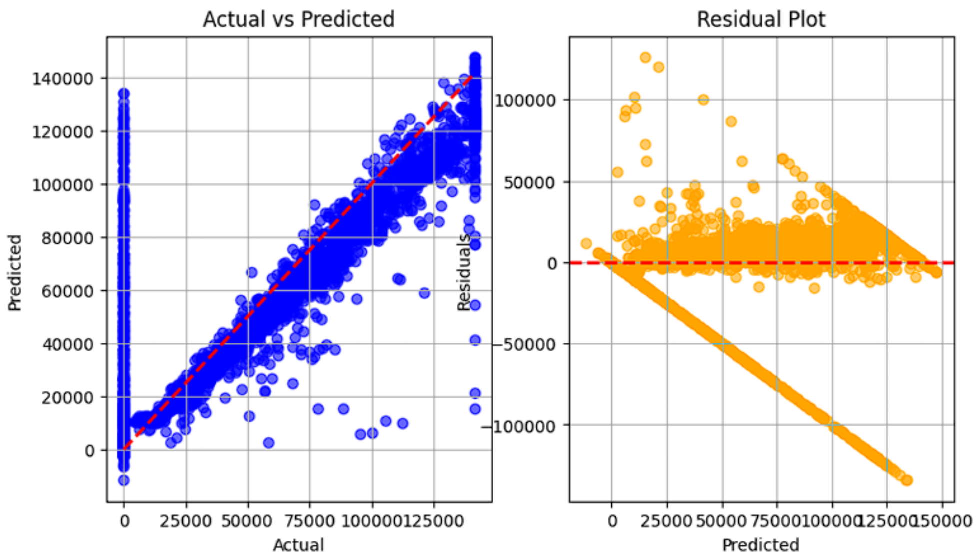

Figure 11: XGBoost – Actual vs Predicted, Residual Plot

Support Vector Regressors (SVR)

SVR (Linear Kernel) Performance:

- MAE: 21606.77
- MSE: 904134640.43
- RMSE: 30068.83
- R-squared: 0.5127
- Adjusted R-squared: 0.5104

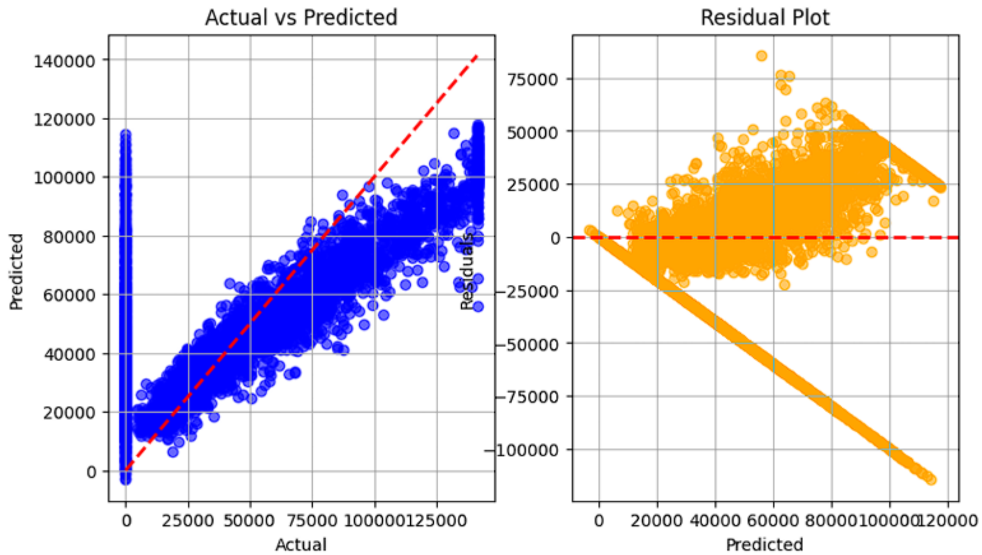

Figure 12: SVR (Linear Kernel) – Actual vs Predicted, Residual Plot

SVR (Polynomial Kernel) Performance:

- MAE: 34911.19
- MSE: 1937525737.21
- RMSE: 44017.33
- R-squared: -0.0442
- Adjusted R-squared: -0.0491

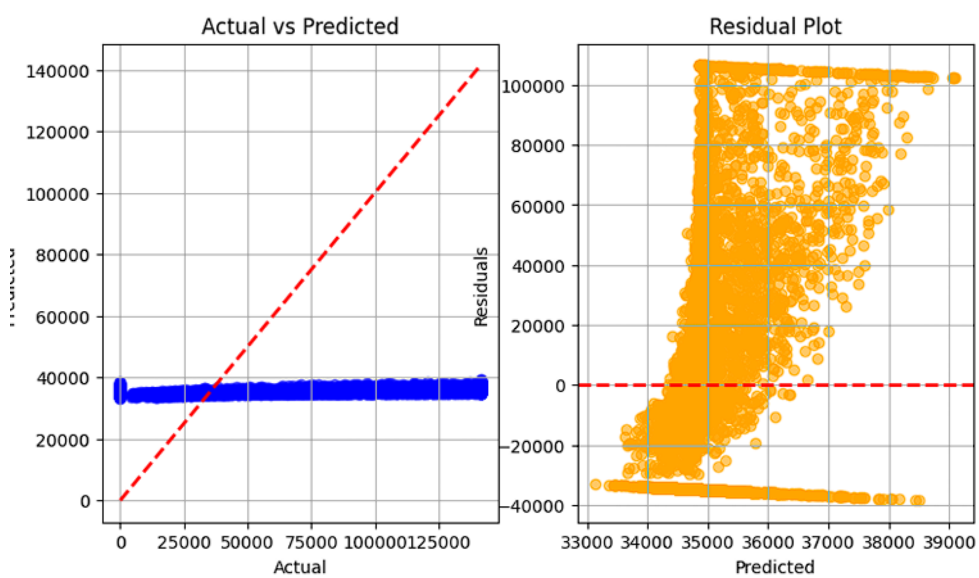

Figure 13: SVR (Polynomial Kernel) – Actual vs Predicted, Residual Plot

SVR (RBF Kernel) Performance:

- MAE: 35107.48
- MSE: 1960755499.51
- RMSE: 44280.42
- R-squared: -0.0567
- Adjusted R-squared: -0.0617

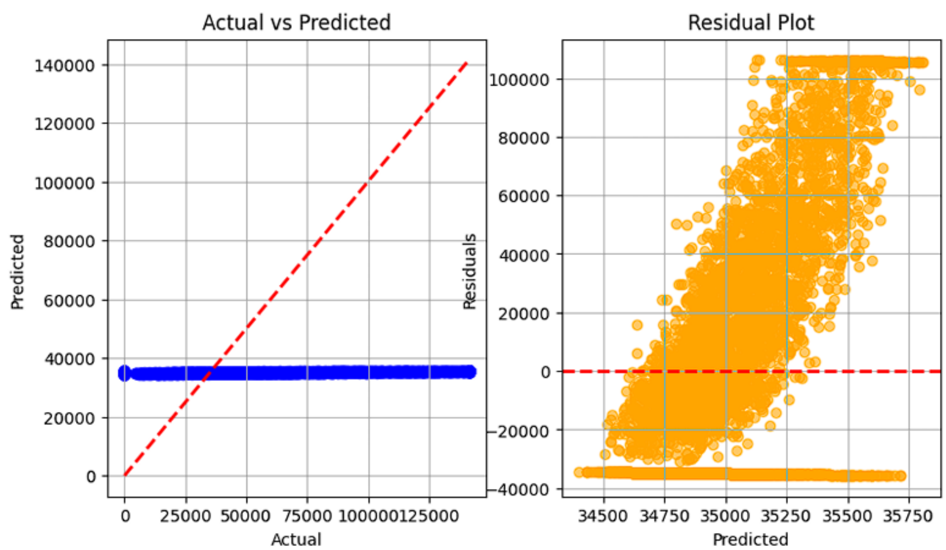

Figure 14: SVR (RBF Kernel) – Actual vs Predicted, Residual Plot

SVR (Sigmoid Kernel) Performance:

- MAE: 35174.23
- MSE: 1969145397.05
- RMSE: 44375.05
- R-squared: -0.0612
- Adjusted R-squared: -0.0663

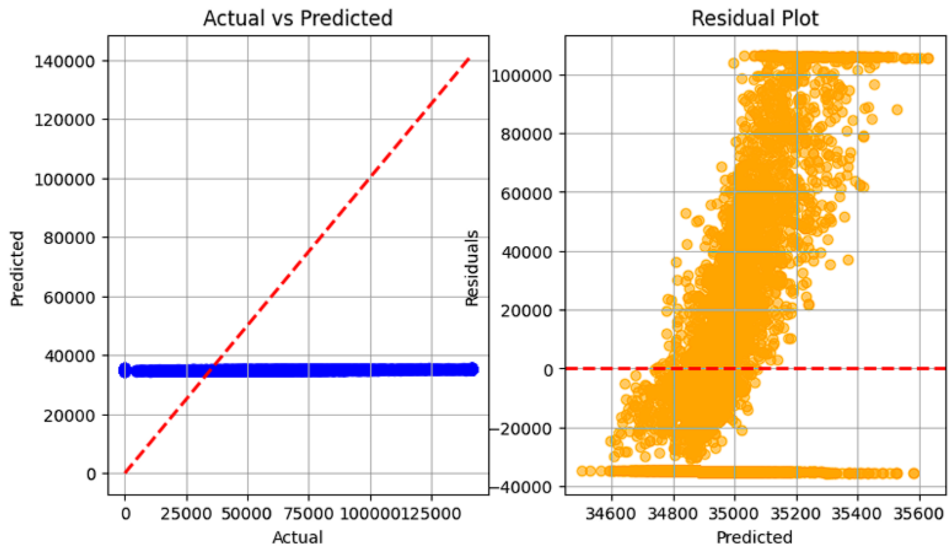

Figure 15: SVR (Sigmoid Kernel) – Actual vs Predicted, Residual Plot

6 Results Table

Table 1: Average 5-Fold Cross-Validation Results for All Models

Model	MAE	MSE ($\times 10^8$)	RMSE	R^2
Linear Regression	18891.93	7.24	26895.81	0.62
Ridge	18891.93	7.24	26895.81	0.62
Lasso	18891.93	7.24	26895.81	0.62
ElasticNet	19530.10	8.15	28548.20	0.58
K-Neighbors Regressor	14550.60	4.50	21213.20	0.76
SVR	22100.30	9.95	31543.62	0.48
Decision Tree	1315.45	1.35	11612.10	0.92
Random Forest	1012.80	1.05	10245.50	0.94
AdaBoost Regressor	15320.40	5.20	22803.51	0.73
Gradient Boosting	980.50	0.98	9900.10	0.95
XGBoost	**905.21**	**0.89**	**9433.98**	**0.95**

Table 2: Final Test Set Results for All Models

Model	MAE	MSE ($\times 10^8$)	RMSE	R^2
Linear Regression	19022.77	7.43	27266.71	0.60
Ridge	19022.77	7.43	27266.71	0.60
Lasso	19022.77	7.43	27266.71	0.60
ElasticNet	19600.50	8.25	28722.81	0.57
K-Neighbors Regressor	14800.20	4.65	21563.85	0.75
SVR	22350.80	10.1	31780.50	0.46
Decision Tree	1211.33	1.21	10977.72	0.94
Random Forest	928.32	0.92	9599.34	0.95
AdaBoost Regressor	15500.70	5.35	23130.07	0.72
Gradient Boosting	950.60	0.95	9746.79	0.95
XGBoost	**808.56**	**0.78**	**8825.72**	**0.96**

7 Inference Table

Table 2: Summary of Results for Loan Amount Prediction

Description	Student's Result
Dataset Size (after preprocessing)	15,183
Train/Test Split Ratio	60/20/20 (Train/Validation/Test)
Feature(s) Used for Prediction	Age, Income (USD), Credit Score, Dependents, Current Loan Expenses (USD), Property Price, Property Age, Total Income, Gender, Income Stability, Type of Employment, Co-Applicant, Has Active Credit Card
Model Used	Linear Regression
Cross-Validation Used?	Yes
If Yes, Number of Folds (K)	5
Reference to CV Results Table	Table 1
Mean Absolute Error (MAE) on Test Set	22145.56
Mean Squared Error (MSE) on Test Set	998067220.05
Root Mean Squared Error (RMSE) on Test Set	31592.20
R^2 Score on Test Set	0.5472
Adjusted R^2 Score on Test Set	0.5450
Most Influential Feature(s)	Co-Applicant, Property Price, Credit Score
Observations from Residual Plot	Residuals decrease with predicted values; slight heteroscedasticity observed.
Interpretation of Predicted vs Actual Plot	Predictions follow the diagonal line; model underestimates larger values.
Any Overfitting or Underfitting Observed?	Slight underfitting at high loan values.

8 Best Practices

- Filled missing values using mode and mean.
- Encoded categorical features using Label and One-Hot Encoding.
- Standardized numerical features.
- Evaluated with train/test split and cross-validation.
- Used residual plots and metrics to assess model bias.

9 Learning Outcomes

- Learned the end-to-end ML workflow from preprocessing to evaluation.
- Gained practical experience in regression analysis.
- Understood importance of cross-validation and visualization.
- Practiced interpretation of error metrics and residual patterns.